EXPERINMENT-28,29

Q Binary tree traversal and count of nodes in binary tree

```
void preorder(node *root){
    if(root ==NULL){
        return;
    }
    printf("%d ",root->key);
    preorder(root->left);
    preorder(root->right);
}
void postorder(node *root){
    if(root==NULL){
        return;
    }
    postorder(root->left);
    postorder(root->right);
    printf("%d ",root->key);

}
void inorder(node *root){
    if(root ==NULL){
        return;
    }
    inorder(root->left);
    printf("%d ",root->key);
    inorder(root->right);
}
int CountNode(node * root){
    if(root ==NULL){
        return 0;
    }
    return 1+ CountNode(root->left) +CountNode(root->right);
}
```

```c
int main()
{
    node *root = NULL;
    root = insert(root,1);
    root = insert(root,2);
    root = insert(root,3);
    root = insert(root,4);
    root = insert(root,5);
    root = insert(root,6);
    root = insert(root,7);
    root = insert(root,8);
    root = insert(root,9);
    printf("the tree element are:\n");
    inorder(root);
    printf("\nthe number of nodes in the tree are:%d",CountNode(root));
    return 0;
    }
```

```
PS D:\languages2\C\l2b2_ha\trees> cd "d:\languages2
the tree element are:
1 2 3 4 5 6 7 8 9
the number of nodes in the tree are:9
```

EXPERINMENT-30

Q WAP TO IMPLEMENT EXPRESSION TREE

```c
#include <stdio.h>
#include <stdlib.h>
struct node {
    char data;
    struct node *left;
    struct node *right;};

struct node *create_node(char data) {
    struct node *new_node = malloc(sizeof(struct node));
    new_node->data = data;
    new_node->left = NULL;
    new_node->right = NULL;
    return new_node;}

struct node *construct_tree(char postfix[]) {
    struct node *stack[100];
    int top = -1;
    int i = 0;
    while (postfix[i] != '\0') {
        char ch = postfix[i];
        if (ch >= 'A' && ch <= 'Z') {
            struct node *new_node = create_node(ch);
            stack[++top] = new_node;
        } else {
            struct node *new_node = create_node(ch);
            new_node->right = stack[top--];
            new_node->left = stack[top--];
            stack[++top] = new_node;}
            i++;}
```

```c
    return stack[top--];}
void inorder(struct node *root) {
    if (root) {
        inorder(root->left);
        printf("%c", root->data);
        inorder(root->right);}}

int main() {
    char postfix[] = "ABC*+D/";
    struct node *root = construct_tree(postfix);
    printf("Inorder traversal of expression tree:\n");
    inorder(root);
    return 0;}
```

OUTPUT

```
PS D:\languages2\C\l2b2_ha\trees> cd "d:\l
{ .\expressiontree }
Inorder traversal of expression tree:
A+B*C/D
```

EXPERIMENT –31,32,33

Binary search Tree

Q WAP to implement binary search insertion, deletion, MIN and MAX funciton

```
#include <stdio.h>
#include <malloc.h>
typedef struct node{
    int key;
    struct node *left;
    struct node *right;
}node;
node *createnewnode(int value){
    node *newNode=(node*)malloc(sizeof(node));
    newNode->key=value;
    newNode->left=NULL;
    newNode->right=NULL;
    return newNode;
}
node *insert(node *root,int value){
    if( root==NULL){
        return createnewnode(value);
    }
    if(root->key <value){
        root->right=insert(root->right,value);
    }
    else if(root->key >value){
        root->left=insert(root->left,value);
    }
    return root;
}
int getRightMin(node *root){
    node *temp=root;
    while(temp->left!=NULL){
        temp=temp->left;}
    return temp->key;
}
```

```
}
node *delete(node *root, int val)
{   if (root==NULL){
        return NULL;}
    else if( root->key <val){
        root->right= delete(root->right,val);}
    else if(root->key>val){
       root->left=  delete(root->left,val);}
    else{
        if(root->left==NULL && root->right==NULL){
            free(root);
            return NULL;}
        else if (root->left==NULL){
            struct node* temp=root->right;
            free(root);
            return temp;}
        else if (root->right==NULL){
            struct node* temp=root->left;
            free(root);
            return temp;}
        else{
           int rightmin;
           rightmin =getRightMin(root->right);
           root->key=rightmin;
           root->right=delete(root->right,rightmin);}}
    return root;}

int leftnodemax(node *root){
    node *temp=root;
    while(temp->right!=NULL){
        temp=temp->right;}
    return temp->key;}
```

```c
int main()
{
    node *root = NULL;
    root = insert(root,1);
    root = insert(root,2);
    root = insert(root,3);
    root = insert(root,4);
    root = insert(root,5);
    root = insert(root,6);
    root = insert(root,7);
    root = insert(root,8);
    root = insert(root,9);
    inorder(root);
    printf("\nthe min value of the trreee-%d\n",getRightMin(root));
    printf("\nthe min value of the trreee-%d\n",leftnodemax(root));
    delete(root,1);
    delete(root,9);printf("the tree after delettion");
    inorder(root);
    return 0;
    }
```

OUTPUT

```
PS D:\languages2\C\l2b2_ha\trees> cd "d:\language
the tree element are:
1 2 3 4 5 6 7 8 9
the min value of the trreee-1

the max value of the trreee-9
```

EXPERIMENT -34,35

Q Heap Insertion and Deletion program

```
#include <stdio.h>
int size = 0;
void swap(int *a, int *b)
    {int temp = *b;
    *b = *a;
    *a = temp;}

void heapify(int array[], int size, int i)
{   if (size == 1)
        { printf("Single element in the heap");}
    else
        {int largest = i;
        int l = 2 * i + 1;
        int r = 2 * i + 2;
        if (l < size && array[l] > array[largest])
            largest = l;
        if (r < size && array[r] > array[largest])
            largest = r;
        if (largest != i)
        {   swap(&array[i], &array[largest]);
            heapify(array, size, largest);}}}

void insert(int array[], int newNum)
{   if (size == 0)
    {   array[0] = newNum;
        size += 1;}
    else
    {array[size] = newNum;
    size += 1;
    for (int i = size / 2 - 1; i >= 0; i--)
    {  heapify(array, size, i);
    }}}
```

```c
void deleteRoot(int array[], int num)
{    int i;
    for (i = 0; i < size; i++)
    {if (num == array[i])
        break;
    }
    swap(&array[i], &array[size - 1]);
    size -= 1;
    for (int i = size / 2 - 1; i >= 0; i--)
    {
        heapify(array, size, i);   }}

void show(int array[], int size)
{  for (int i = 0; i < size; ++i)
    printf("%d ", array[i]);
    printf("\n");}

int main()
{   int array[10];
    insert(array, 3);
    insert(array, 4);
    insert(array, 9);
    insert(array, 5);
    insert(array, 2);
    printf("Max-Heap array: ");
    show(array, size);
    deleteRoot(array, 4);
    printf("After deleting an element: ");
    show(array, size);
}
```

Output-

```
PS D:\languages2\C> cd  d:\languages2\C\peoblem
Max-Heap array: 9 5 4 3 2
After deleting an element: 9 5 2 3
PS D:\languages2\C\peoblem solbi\trees>
```

EXPERINMENT-36

Q Heap sort program

```
#include <stdio.h>
void swap(int *a,int *b)
{   int temp = *a;
    *a = *b;
    *b = temp;}

void Heapify(int arr[],int index,int size){
    int left = 2 * index + 1;
    int right = left + 1;
    int max = index;
    if(left <= size && arr[left] > arr[max])
        max = left;
    if(right <= size && arr[right] > arr[max])
        max = right;
    if(index != max)
    {   swap(&arr[max],&arr[index]);
        Heapify(arr,max,size); }}

void BuildHeap(int arr[],int size)
{   int i = size / 2;
    for(; i >= 0; i--)
        Heapify(arr,i,size);}

void HeapSort(int arr[],int size)
{BuildHeap(arr,size);
    while(size > 0)
    {int t = arr[0];
        arr[0] = arr[size];
        arr[size] = t;
        size--;
        Heapify(arr,0,size);}}
```

```c
void main()
{
    int n = 7;
    int arr[7] = {80, 30, 60, 10, 20, 40, 50};

    printf("Array in sorted order : ");
    HeapSort(arr,n-1);

    for(int i = 0; i < 7; i++)
        printf("%d ",arr[i]);
}
```

OUTPUT

```
re-enable it, run 'Import-Module PSReadLine'.

PS D:\languages2\C> cd "d:\languages2\C\l2b2_ha\trees\" ;
Array in sorted order : 10 20 30 40 50 60 80
PS D:\languages2\C\l2b2_ha\trees>
```

EXPERIMENT- 37

Q Adjacency matrix representation of GRAPH

```
#include <stdio.h>
#define V 4
void init(int arr[][V]){
    for(int i=0;i<V;i++){
        for(int j=0;i<V;i++){
            arr[i][j]=0;}}}

void addEdge(int arr[V][V],int src,int dest){
    arr[src][dest]=1;}

void show(int arr[V][V]){
    for(int i=0;i<V;i++){
        for(int j=0;j<V;i++){
            printf("%d  ",arr[i][j]);
            }
        printf("\n"); }}

int search(int arr[][V],int src,int dest){
    if(arr[src][dest]==1)
        return 1;
    return 0;}

void removeEdge(int arr[][V],int src,int dest){
    arr[src][dest]=0;}

void takeGraph(int adj[][V])
{   int vertex,edges;
    printf("enter total vertex:");
    scanf("%d",&vertex);
```

```c
    printf("enter the total number of edges:");
    scanf("%d",&edges);
    int u,v;
    for(int i=0; i<edges; i++){
        printf("Enter %d Edge(u,v): ",i+1);
        scanf("%d %d",&u,&v);
            addEdge(adj,u,v);
    }}

int main()
{
    int adjMatrix[V][V];
    init(adjMatrix);
    takeGraph(adjMatrix);

    printf("Adjacency Matrix Representation of the Graph\n");
    show(adjMatrix);
    return 0;
}
```

Output

```
Adding Edge From 0 to 1
Adding Edge From 0 to 2
Adding Edge From 0 to 3
Adding Edge From 1 to 3
Adding Edge From 1 to 4
Adding Edge From 2 to 3
Adding Edge From 3 to 4
Adjacency Matrix Representation of the Graph
0 1 1 1 0
0 0 0 1 1
0 0 0 1 0
0 0 0 0 1
0 0 0 0 0
```

EXPERINMENT-37

Q Adjacency list representation of GRAPH

```
#include<stdio.h>
#include<stdlib.h>
#define V 5

struct node
{   int data;
    struct node *next;};

struct node *adjList[V];
void init(){
    int i;
    for(i = 0; i < V; i++)
        adjList[i] = NULL;   }

int hasEdge(int src, int dest)
{   struct node *temp = adjList[src];
    while(temp != NULL)
    {   if(temp->data == dest)
            return 1;
        temp = temp->next;}
    return 0;}

void removeEdge(int src, int dest)
{   if(adjList[src] == NULL)
        return;
    if(adjList[src]->data == dest)
    {   struct node *temp = adjList[src];
        adjList[src] = adjList[src]->next;
        free(temp);}
```

```
    else
    {struct node *current = adjList[src], *temp;
        while(current->next != NULL)
        {if(current->next->data == dest)
            {    temp = current->next;
                current->next = temp->next;
                free(temp);
                break;}
            else{
                current = current->next;}
        }}}

void addEdge(int src, int dest)
{   struct node* newNode = malloc(sizeof(struct node));
    newNode->data = dest;
    newNode->next = NULL;
    if(adjList[src] == NULL)
        adjList[src] = newNode;
    else
    {   struct node *last = adjList[src];
        while(last->next != NULL)
        {   last = last->next;}
        last->next = newNode;}}

void printGraph()
{   int i;
    for(i = 0; i < V; i++)
    {   struct node *temp = adjList[i];
```

```
        printf("adjList[%d] -> ",i);
        while(temp != NULL)
        {   printf("%d->",temp->data);
            temp = temp->next;}
        printf("NULL\n");
    }}
int main()
{   init();
    printf("Adding Edge From 0 to 1\n");
    addEdge(0,1);
    printf("Adding Edge From 0 to 2\n");
    addEdge(0,2);
    printf("Adding Edge From 0 to 3\n");
    addEdge(0,3);
    printf("Adding Edge From 1 to 3\n");
    addEdge(1,3);
    printf("Adding Edge From 1 to 4\n");
    addEdge(1,4);
    printf("Adding Edge From 2 to 3\n");
    addEdge(2,3);
    printf("Adding Edge From 3 to 4\n");
    addEdge(3,4);
    printf("Adjacency List Representation of the Graph\n"
    printGraph();
    printf("Removing Edge(0, 2)..\n");
    removeEdge(0,2);
    return 0;
}
```

```
Adding Edge From 0 to 1
Adding Edge From 0 to 2
Adding Edge From 0 to 3
Adding Edge From 1 to 3
Adding Edge From 1 to 4
Adding Edge From 2 to 3
Adding Edge From 3 to 4
Adjacency List Representation of the Graph
adjList[0]->1->2->3->NULL
adjList[1]->3->4->NULL
adjList[2]->3->NULL
adjList[3]->4->NULL
adjList[4]->NULL
Removing Edge(0, 2)..
```

EXPERINMENT-38

Q Graph BFS program

```
#include<stdio.h>
#define SIZE 100
int queue[SIZE];
int front,rear;

void takeGraph(int adj[SIZE][SIZE], int vertex, int isDirected)
{    int edges,u,v;
    printf("Enter total number of Edges: ");
    scanf("%d",&edges);
    for(int i=0; i<edges; i++)
    {   printf("Enter %d Edge(u,v): ",i+1);
        scanf("%d %d",&u,&v);
            adj[u][v] = 1;
        if(!isDirected)
          adj[v][u] = 1;}}

void showGraph(int adj[SIZE][SIZE], int v)
{   printf("\nGraph is\n");
for(int i=1; i<=v; i++)
    {   for(int j=1; j<=v; j++)
        {    printf("%-3d ",adj[i][j]);}
        printf("\n");}}

void enqueue(int value)
{   if(rear == SIZE -1)
    {   printf("Queue overflow\n");
        return;}
    rear = rear+1;
    if(front == -1)
        front = 0;
    queue[rear] = value;}
```

```
int dequeue()
{   if(front == -1)
        return -1;
    if(front == rear)
    {   int temp =  queue[front];
        front = rear = -1;
        return temp;}
    else
    return queue[front++];}

int isEmpty()
{    return front == -1;}

void BFS(int adj[SIZE][SIZE], int v,int status[], int start)
{    front = rear = -1;
    enqueue(start);
    while(!isEmpty())
    {
        int u = dequeue();
        status[u] = 2; // visited
        printf("%d, ",u);
        for(int i=1; i<=v; i++)
        {
            if(adj[u][i] == 1)
            {
                if(status[i] == 0)
                {
                    enqueue(i);
                    status[i] = 1; //process
                    }}}}}
```

```c
int main()
{
    int adj[SIZE][SIZE] = {0};
    int status[SIZE] = {0};
    int isDirected;
    int v;
    printf("Enter number of vertex: ");
    scanf("%d",&v);
    printf("Directed/Indirected(1/0):");
    scanf("%d",&isDirected);
    takeGraph(adj,v,isDirected);
    showGraph(adj,v);
    printf("\nBFS of the graph:\n");
    for(int i=1; i<=v; i++)
    {
        if(status[i]==0)
            BFS(adj,v,status,i);
    }
    printf("\n");
    for(int i=1; i<=v; i++)
    {
        printf("%d ",status[i]);
    }
    printf("front=%d, rear = %d",front,rear);
}
```

```
Enter number of vertex: 7
Directed/Undirected (1/0): 1
Enter the number of edges: 10
Enter 1 edge(u,v): 1 2
Enter 2 edge(u,v): 1 4
Enter 3 edge(u,v): 2 5
Enter 4 edge(u,v): 3 1
Enter 5 edge(u,v): 3 4
Enter 6 edge(u,v): 4 5
Enter 7 edge(u,v): 6 5
Enter 8 edge(u,v): 7 1
Enter 9 edge(u,v): 7 3
Enter 10 edge(u,v): 7 6
0  1  0  1  0  0  0
0  0  0  0  1  0  0
1  0  0  1  0  0  0
0  0  0  0  1  0  0
0  0  0  0  0  0  0
0  0  0  0  1  0  0
1  0  1  0  0  1  0

BFS of the graph is: 1, 2, 4, 5, 3, 6, 7,
```

EXPERINMENT-39

Q Graph DFS program

=

```
#include<stdio.h>
#define SIZE 100
int stack[SIZE];
int top;
void takeGraph(int adj[SIZE][SIZE], int vertex, int isDirected)
{   int edges,u,v;
    printf("Enter total number of Edges: ");
    scanf("%d",&edges);
    for(int i=0; i<edges; i++){
        printf("Enter %d Edge(u,v): ",i+1);
        scanf("%d %d",&u,&v);
            adj[u][v] = 1;
        if(!isDirected)
          adj[v][u] = 1;}}

void showGraph(int adj[SIZE][SIZE], int v)
{   printf("\nGraph is\n");
    for(int i=1; i<=v; i++)
    {for(int j=1; j<=v; j++)
        {printf("%-3d ",adj[i][j]);}
        printf("\n");}}

void push(int value)
{   if(top == SIZE -1)
    {printf("stack overflow\n");
        return;}
    top = top+1;
    stack[top] = value;
}
int pop()
{   if(top == -1)
```

```
        return -1;
    return stack[top--];}

int isEmpty()
{   return top == -1;}

void DFS(int adj[SIZE][SIZE], int v,int status[], int start)
{   top = -1;
    push(start);
    status[start]=1;
    while(!isEmpty())
    {   int u = pop();
        status[u] = 2; // visited
        printf("%d, ",u);
        for(int i=1; i<=v; i++)
        {   if(adj[u][i] == 1)
            {   if(status[i] == 0)
                {   push(i);
                    status[i] = 1; //process
                    }}}}}

int main()
{
    int adj[SIZE][SIZE] = {0};
    int status[SIZE] = {0};
    int isDirected;
    int v;
    printf("Enter number of vertex: ");
    scanf("%d",&v);
    printf("Directed/Indirected(1/0):");
    scanf("%d",&isDirected);
    takeGraph(adj,v,isDirected);
    showGraph(adj,v);
    printf("\nD0.FS of the graph:\n");
    for(int i=1; i<=v; i++)
    {
        if(status[i]==0)
            DFS(adj,v,status,i);
    }
    printf("\n");
    for(int i=1; i<=v; i++)
    {
        printf("%d ",status[i]);
    }
    printf("top = %d",top);
}
```

Output

```
Enter number of vertex: 7
Directed/Undirected (1/0): 1
Enter the number of edges: 10
Enter 1 edge(u,v): 1 2
Enter 2 edge(u,v): 1 4
Enter 3 edge(u,v): 2 5
Enter 4 edge(u,v): 3 1
Enter 5 edge(u,v): 3 4
Enter 6 edge(u,v): 4 5
Enter 7 edge(u,v): 6 5
Enter 8 edge(u,v): 7 1
Enter 9 edge(u,v): 7 3
Enter 10 edge(u,v): 7 6
0  1  0  1  0  0  0
0  0  0  0  1  0  0
1  0  0  1  0  0  0
0  0  0  0  1  0  0
0  0  0  0  0  0  0
0  0  0  0  1  0  0
1  0  1  0  0  1  0

DFS of the graph is: 1, 4, 5, 2, 3, 6, 7,
```

EXPERINMENT-40

Q MST using Prim's method

```
#include <stdio.h>
#define MAX 9999
#define SIZE 100

void takeGraph(int adj[SIZE][SIZE], int vertex)
{   int edges,u,v, w;
    printf("Enter total number of Edges: ");
    scanf("%d",&edges);
    for(int i=0; i<edges; i++)
    {   printf("Enter %d Edge(u,v) and weight: ",i+1);
        scanf("%d %d %d",&u,&v,&w);
            adj[u][v] = w;
            adj[v][u] = w;}}

void showGraph(int adj[SIZE][SIZE], int v)
{   int sum = 0;
    printf("\nMST is\n");
    for(int i=1; i<=v; i++)
    {   for(int j=1; j<=v; j++)
        {   printf("%-3d ",adj[i][j]);
            sum += adj[i][j];}
        printf("\nFinal Value=%d",sum);}}

int minKey(int key[], int status[], int v){
    int min = MAX, min_index;
    for (int i = 1; i <= v; i++)
        if (status[i] == 0 && key[i] < min)
        {   min = key[i];
            min_index = i;}
    return min_index;}
```

```c
int showTree(int parent[], int adj[SIZE][SIZE], int v)
{   printf("Edge \tWeight\n");
    for (int i = 1; i <= v; i++)
        printf("%d - %d \t%d \n", parent[i], i,
            adj[i][parent[i]]);}

void prim_MST(int adj[SIZE][SIZE], int v)
{   int parent[SIZE];
    int key[SIZE];
    int status[SIZE] = {0};
    for (int i = 1; i <= v; i++)
    {   key[i] = MAX;
        status[i] = 0;}
    key[1] = 0;
    parent[1] = -1;
    for (int count = 1; count <=v; count++) {
        int a = minKey(key, status,v);
        status[a] = 1;
        for (int b = 1; b <=v; b++)
            if (adj[a][b]!=0 && status[b] == 0 && adj[a][b] < key[b])
                {   parent[b] = a;
                    key[b] = adj[a][b];}}
    showTree(parent, adj,v);}

int main()
{   int adj[SIZE][SIZE] = {0};
    int status[SIZE] = {0};
    int v;
    printf("Enter number of vertex: "); scanf("%d",&v);
    takeGraph(adj,v);   showGraph(adj,v);   prim_MST(adj,v);    return 0;}
```

OUTPUT

```
Enter number of vertex: 7
Enter total number of Edges: 10
Enter 1 Edge(u,v) and weight: 1 2 6
Enter 2 Edge(u,v) and weight: 1 4 1
Enter 3 Edge(u,v) and weight: 2 5 1
Enter 4 Edge(u,v) and weight: 3 1 2
Enter 5 Edge(u,v) and weight: 3 4 3
Enter 6 Edge(u,v) and weight: 4 5 4
Enter 7 Edge(u,v) and weight: 6 5 2
Enter 8 Edge(u,v) and weight: 7 1 3
Enter 9 Edge(u,v) and weight: 7 3 2
Enter 10 Edge(u,v) and weight: 7 6 6

MST is
0   6   2   1   0   0   3
Final Value=126   0   0   0   1   0   0
Final Value=192   0   0   3   0   0   2
Final Value=261   0   3   0   4   0   0
Final Value=340   1   0   4   0   2   0
Final Value=410   0   0   0   2   0   6
Final Value=493   0   2   0   0   6   0
Final Value=60Edge      Weight
-1 - 1  0
5 - 2   1
1 - 3   2
1 - 4   1
4 - 5   4
5 - 6   2
3 - 7   2
```